A Supplementary file on the Flapping-Wing Robotic Platform and Experimental Implementation of Natural Hovering Extremum Seeking

Ahmed A. Elgohary, Rohan Palanikumar, Simone Martini, and Sameh A. Eisa

Abstract

This supplementary note provides a tutorial-style description of the experimental setup used for the demonstration of Natural Hovering Extremum Seeking (NH-ES) on a physical flapping-wing robotic platform. The goal is to provide practical details about the Flapper Nimble+ robot, the actuation system, the sensing architecture, and the objective-function measurements used in the experiments. In particular, this note describes the motor and servo control inputs, the OptiTrack motion-capture system used for the set-altitude-objective experiment, and the light-sensor setup used for the light-source objective experiment. These details complement the main manuscript and help clarify how the model-free NH-ES feedback law is realized experimentally under practical conditions that naturally include sensing noise, measurement bias, wireless communication delay, computation delay, and actuator response delay.

1 Purpose of this supplementary note

The purpose of this supplementary note is to provide additional practical details about the flapping-wing micro air vehicle (FWMAV)-type experimental platform used for the Natural Hovering Extremum Seeking (NH-ES) demonstration. While the main manuscript focuses on the NH-ES concept, theoretical development, and experimental results, this note focuses on the physical experimental setup and the main hardware/software components used to implement the controller.

In particular, this supplementary note helps readers understand the Flapper Nimble+ robotic platform, the motor and servo actuation inputs, and the sensing systems used to obtain the objective-function measurements. For the altitude-objective experiment, the feedback signal is obtained from the measured vertical position provided by the OptiTrack motion-capture system. For the light-source objective experiment, the feedback signal is obtained from an analog light sensor mounted on the robot and transmitted wirelessly through an Arduino Nano to a Python-based interface.

2 Flapping robot and mechanical structure

The experimental platform used in this work is based on the Flapper Nimble+ robotic platform, which is a bioinspired flapping-wing development platform designed for research, education, and entertainment applications. The platform was purchased from Flapper Drones, and additional information about the platform can be found on the official product page: https://flapper-drones.eu/nimbleplus/.

The Flapper Nimble+ is a lightweight flapping-wing robotic system with a wingspan of approximately 49 cm. The platform is designed to produce insect-like flight using flapping wings rather

than conventional propellers. This makes the system especially relevant for studying bioinspired flight, flapping-wing aerodynamics, flight control, and autonomous flight. In the present work, the platform is used as the physical robotic system for implementing the proposed Natural Hovering Extremum Seeking (NH-ES) controller .

The main body of the robot consists of a lightweight frame, two flapping wings, two brushless DC motors, two rotary servos, a flight-control board, electronic speed controllers (ESCs), a battery, and expansion interfaces for additional sensors or modules. According to the official specifications, the platform has a recommended maximum payload of 25 g, a minimum take-off weight of approximately 102 g, and a recommended maximum take-off weight of approximately 127 g. The robot can be powered either by a 2S, 300 mAh LiPo battery or by an external power supply. In this experiment, the external power-supply option was used instead of the battery. The supply voltage was set to 8.4 V, which corresponds to the fully charged voltage of a 2S LiPo battery. The current requirement was at least 5 A; however, the current limit was set to 8 A with over-current protection enabled. This setup was selected to maintain a consistent voltage and current level during the experiment and to avoid the voltage drop that can occur when using a battery over time.

The flapping mechanism is actuated using two brushless DC motors, while two coreless rotary servos are used for wing-control and maneuvering functions. The nominal flapping frequency is approximately 12 Hz during hover and can reach approximately 20 Hz at maximum throttle. The robot uses a Crazyflie Bolt 1.1 flight controller, and its onboard architecture supports open-source firmware, a Python API, live telemetry, onboard data logging through a microSD card slot, and wireless communication through an nRF51822-based interface with Crazyradio.

Figure 1 shows the Flapper Nimble+ platform used in the experimental setup. This figure is included to identify the main physical platform used in the experiments and to show the overall body, wing, and landing-gear configuration.



Figure 1: Flapper Nimble+ flapping-wing robotic platform used in the NH-ES experimental setup. The platform is a lightweight bioinspired flying robot with two flapping wings, two brushless DC motors, two rotary servos, a Crazyflie Bolt 1.1 flight controller, and wireless communication capability.

3 Actuation system and control inputs

The Flapper Nimble+ platform uses two main actuation mechanisms: the flapping actuation generated by the brushless DC motors and the wing pitch control actuation generated by the rotary servos. In the experimental implementation, these actuation channels are used to realize the NH-ES control input applied to the physical flapping-wing robot.

The control input vector is written as

$$\boldsymbol{u} = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}, \tag{1}$$

where u_1 denotes the command associated with the flapping motor actuation, and u_2 denotes the command associated with the pitch servo actuation. The exact low-level mapping between these command variables and the onboard motor/servo signals is handled by the robot electronics and firmware. Therefore, from the control-design perspective used in this work, the flapping-wing robot is treated as a black-box, nonlinear system of the form

$$\dot{x} = f(x, \boldsymbol{u}), \tag{2}$$

or equivalently,

$$\dot{x} = f(x, u_1, u_2), \tag{3}$$

where x denotes the state of the physical robot.

The NH-ES implementation does not require an explicit analytical model of the robot dynamics. Instead, the controller updates the input commands using the measured objective-function signal. In the experiment, both actuation channels use the same adaptive estimate $\hat{u}$, but with different scaling and excitation gains. The commanded inputs are written as

$$u_1 = c_1\hat{u} + a_1\omega\cos(\omega t), \qquad u_2 = c_2\hat{u} + a_2\omega\cos(\omega t), \tag{4}$$

where c_1 and c_2 are input scaling constants, a_1 and a_2 are excitation amplitudes, and ω is the excitation frequency. The adaptive estimate is updated according to

$$\dot{\hat{u}} = kJ\omega\cos(\omega t), \tag{5}$$

where J is the measured objective function and k is the learning rate.

This input structure allows the experimental controller to modify the motor and servo commands using only the measured value of the objective function. As a result, the implementation remains model-free and does not require explicit knowledge of the aerodynamic forces, flapping-wing mechanics, or full nonlinear equations of motion of the robot.

4 Camera system and motion tracking

In the experiment presented in the "First experiment: set altitude-objective" subsection of the main manuscript, the OptiTrack Motion Capture System (MCS) is used to obtain the feedback measurement required for the NH-ES implementation. In particular, the MCS provides the measured vertical position, or z-position, of the flapping-wing robot. This altitude measurement is used to construct the objective function for the set-altitude-seeking experiment.

The experimental setup uses an OptiTrack MCS together with the Motive software to track the motion real-time of the flapping-wing robotic platform inside the experimental area. The MCS

provides accurate position measurements of the robot during the experiment and allows the robot motion to be monitored and recorded while the NH-ES controller is running.

The motion-capture system consists of multiple OptiTrack cameras distributed around the experimental area. In the laboratory setup, thirteen cameras are used, including ten Prime 17W cameras and three Prime^{X} 22 cameras. This number of cameras provides good coverage of the experimental space and improves the visibility of the reflective markers attached to the robot. For smaller experimental areas, the same tracking principle can be implemented with fewer cameras, provided that the markers remain visible to a sufficient number of cameras during the experiment.



Figure 2: OptiTrack Prime^{X} 22 camera used as part of the motion-capture system. The cameras detect reflective markers attached to the flapping-wing robot and provide position measurements through the Motive software.

The OptiTrack cameras track the robot by detecting reflective markers attached to the body of the flapping-wing platform. Before running the experiment, the camera system is calibrated in Motive. During this process, reflective objects in the environment, or reflective items worn by users, may introduce tracking errors because they can be detected as unwanted markers. Therefore, these reflections should be removed from the experimental area or masked inside the Motive software before calibration.

After calibration, the reflective markers attached to the flapping robot are selected in Motive and grouped to define a rigid body. The position of this rigid body is then used as the measured position of the robot. At least three reflective markers are required to create a rigid body in Motive; however, using four or more markers can improve the robustness and stability of the tracking. If the orientation of the robot is also required, the markers should be placed in an asymmetric configuration so that Motive can distinguish the body orientation more reliably.

Typically, at least two cameras must observe a marker for the system to reconstruct its three-dimensional position. If higher tracking accuracy is needed, this requirement can be adjusted in the Motive settings. In addition, the pixel and circle filters in Motive can be tuned to help distinguish the intended reflective markers from other reflective objects in the environment.

Once the rigid body is created and the tracking is verified, Motive can stream the rigid-body data to the control computer. In the experimental setup, the PC running Motive is connected to a local router using an Ethernet cable. The local interface in Motive should be set to the IP address

of the PC connected to this router. This IP address can be found on a Windows device by using the *ipconfig* command. It is important not to select the loopback address or the camera-network address as the local interface. The computer running the Simulink control loop is also connected to the same router through Ethernet. In this setup, the rigid-body data are streamed using a unicast transmission method, where a single listener receives the motion-capture data.

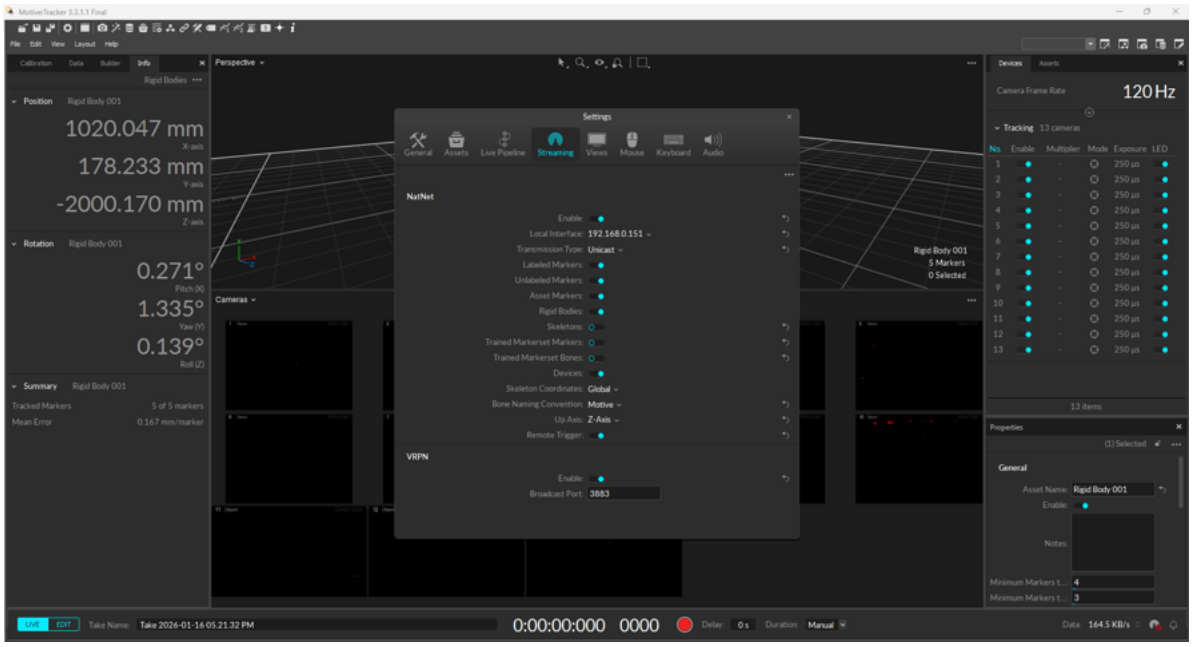


Figure 3: Motive software interface used to define the robot as a rigid body and stream the rigid-body position measurements. In the altitude-objective experiment, the streamed z-position is used to construct the feedback objective function for the NH-ES controller.

The camera frame rate determines how many position measurements are available per second. Although an extremely high frame rate is not always required, the motion-capture system should provide at least one valid position sample per control-loop time step. This ensures that the controller and data-logging system receive updated motion information during the experiment. Finally, rigid-body streaming should be enabled in Motive, and the up-axis should be set consistently with the experimental coordinate convention, typically with the z-axis defined as the vertical direction.

5 Light sensor and objective-function measurement

In all moth-like light-source-seeking experiments presented in Sections III and IV of the main manuscript, the feedback measurement used by the NH-ES controller is obtained from a light sensor mounted on the flapping-wing robot. Unlike the set-altitude-objective experiment, where the feedback signal is provided by the motion-capture system through the measured vertical position, in this experiment the objective-function measurement is obtained directly from the measured light intensity. Therefore, the light sensor serves as the main sensing element for the moth-like, source-seeking implementation.

An analog light sensor is used to measure the incident light intensity. This type of sensor is inexpensive and has been used successfully in previous light-source-seeking experiments (e.g.,[1, 2, 3]). The sensing element consists of a photoresistive component whose resistance changes depending

on the amount of light incident on its surface. In general, the sensor output reaches its maximum response when the light intensity incident on the sensing surface is greatest. The sensor also includes an adjustable sensitivity module, which allows the user to tune the response of the sensor according to the experimental environment.

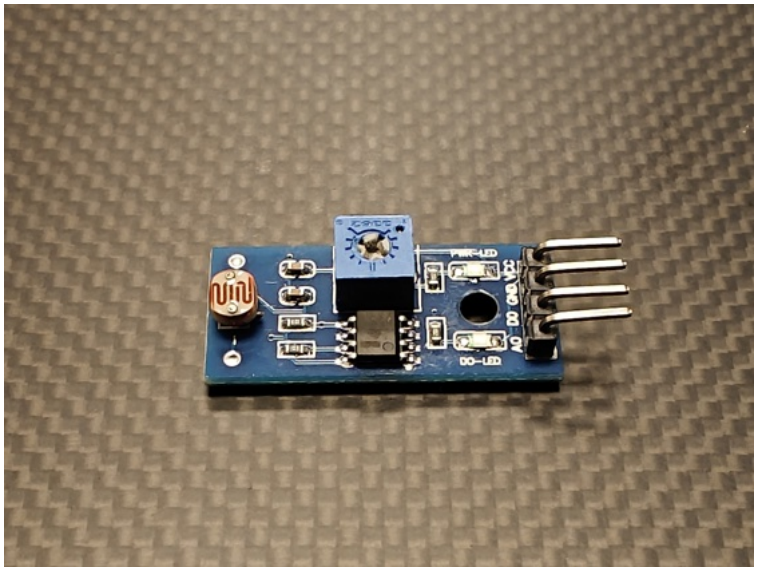


Figure 4: Analog light sensor used to measure light intensity in the light-source NH-ES experiment.

The light sensor is interfaced with an Arduino Nano board, which is used to read the sensor output and transmit the measured signal wirelessly. In particular, the sensor is connected to the Arduino Nano through three basic electrical connections: the sensor VCC pin is connected to the Arduino power supply, the sensor GND pin is connected to the Arduino ground, and the sensor analog output pin is connected to one of the Arduino analog input pins. The Arduino Nano then acquires the light-sensor reading and transmits it through a WiFi connection.

In the experimental implementation, the transmitted sensor reading is received on a Python-based interface running on the external computer. This received signal is then used as the objective-function measurement J for the NH-ES algorithm. Therefore, the controller does not require an explicit analytical model of the light field. Instead, the measured light intensity is used directly in real time to update the adaptive control law.

This sensing architecture allows the flapping-wing robot to perform light-source-seeking behavior in a fully model-free manner. The light sensor provides the environmental measurement, the Arduino Nano handles data acquisition and wireless transmission, and the external Python interface receives the measurement and supplies it to the NH-ES control loop as the objective-function signal. As in any physical implementation, this measurement channel may include sensor noise, ambient-light effects, sampling effects, and wireless transmission delays, which are naturally present in the experimental setup.

6 Discrete-time implementation and rate-transition architecture

The experimental NH-ES implementation is carried out in discrete time because the sensing, communication, computation, and actuation processes operate at finite sampling rates. Therefore, the continuous-time NH-ES update law is implemented numerically in the Python-based control inter-

face. The measured objective-function signal is first obtained from the analog light sensor, read by the Arduino Nano, and transmitted wirelessly to the external computer. The Python receiver then receives this measurement, evaluates the NH-ES control law, and sends the corresponding motor and servo commands to the Flapper Nimble+ board.

Figure 5 shows the simplified control and communication architecture used in the light-source-seeking experiments. The first rate-transition block represents the transition between the light-sensor/Arduino measurement rate and the Python receiver/control update rate. This step is important because the sensor measurements are received through wireless communication and may not arrive at exactly the same rate as the control-loop update. The second rate-transition block represents the transition between the Python control-command update rate and the command transmission rate to the flapper board, where the motor and pitch-servo actuation commands are applied.

In this implementation, the Python interface performs the main control computation. It receives the measured light-intensity signal J, updates the NH-ES control law, and generates the two command signals u_1 and u_2, where u_1 denotes the flapping motor command and u_2 denotes the pitch servo command. These commands are then transmitted to the flapper board and applied to the physical robot. Since the implementation is performed in real time, small discretization effects, sampling mismatch, wireless communication delay, and command-update delay are naturally present in the system. These effects are part of the practical experimental implementation and are expected in a wireless robotic setup.

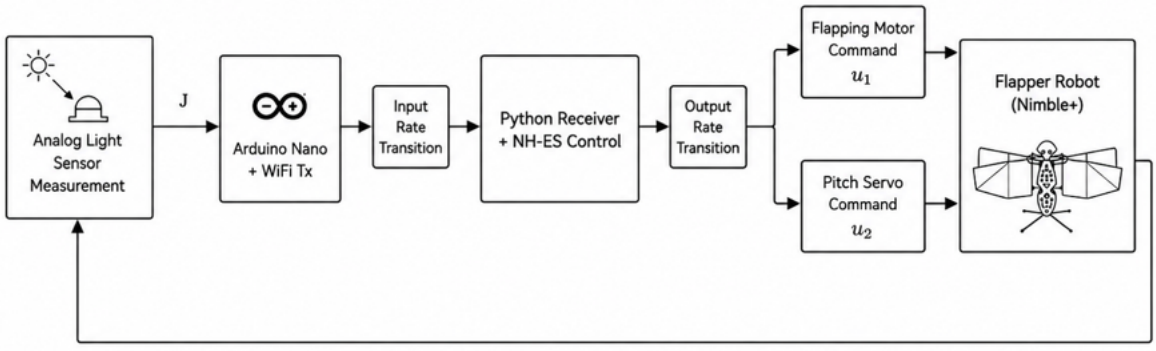


Figure 5: Simplified discrete-time control and communication architecture for the light-source-seeking NH-ES experiment. The analog light sensor measurement is read by the Arduino Nano and transmitted wirelessly to the Python receiver, where the NH-ES control law is implemented. The first rate-transition block represents the transition from the sensor/Arduino transmission rate to the Python control update rate, while the second rate-transition block represents the transition from the Python command update rate to the flapper-board actuation rate.

7 Concluding remarks

This supplementary note provided practical details about the experimental setup used to implement Natural Hovering Extremum Seeking (NH-ES) on the Flapper Nimble+ robotic platform. The note described the flapping-wing robot body, the motor and servo actuation inputs, the motion-capture system used for the altitude-objective experiment, and the light-sensor setup used for the light-source objective experiment.

In both experiments, the NH-ES controller uses measured objective-function signals directly from the physical setup, without requiring an explicit analytical model of the robot dynamics or

the environment. The implementation also naturally includes practical nonidealities such as sensor noise, measurement bias, wireless communication delay, computation delay, and actuator response delay. These effects are expected in real robotic experiments and highlight the practical nature of the NH-ES implementation.

Overall, this note clarifies how the proposed model-free NH-ES feedback law is realized experimentally on a physical flapping-wing robotic platform.

References

[1] Shivam Bajpai, Ahmed A Elgohary, and Sameh A Eisa. Model-free source seeking by a novel single-integrator with attenuating oscillations and better convergence rate: Robotic experiments. In *2024 European Control Conference (ECC)*, pages 472–479. IEEE, 2024.

[2] Ahmed A Elgohary, Sameh A Eisa, and Shivam Bajpai. Model-free and real-time unicycle-based source seeking with differential wheeled robotic experiments. *IFAC-PapersOnLine*, 59(30):25–30, 2025.

[3] Rohan Palanikumar, Ahmed A Elgohary, Victoria Grushkovskaya, and Sameh A Eisa. Model-free source seeking of exponentially convergent unicycle: theoretical and robotic experimental results. *IFAC Journal of Systems and Control*, 35:100378, 2026.